

LA PROGRAMMAZIONE AD OGGETTI

prof.ssa Taffurelli

LA PROGRAMMAZIONE STRUTTURATA

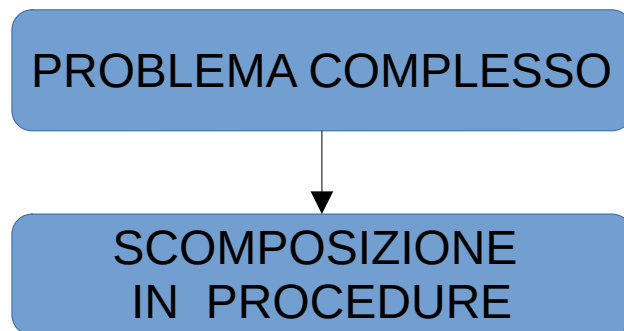
- Definizione del problema e dei dati di input e di output
- Organizzazione algoritmo
 - sviluppato sotto forma di procedure e funzioni
 - Uso rigoroso delle tre strutture fondamentali: Sequenza , Selezione, Iterazione
- Stesura programma nel linguaggio di programmazione
- Test e verifica:
 - Esecuzione del programma con casi di prova
 - Individuazione e correzione degli errori
 - Validazione dei risultati

=> DEFINIZIONE: La programmazione strutturata produce programmi ordinati e modulari, basati sull'uso rigoroso di strutture di controllo. Il codice risulta chiaro, leggibile e organizzato in sequenze logiche di procedure e funzioni.

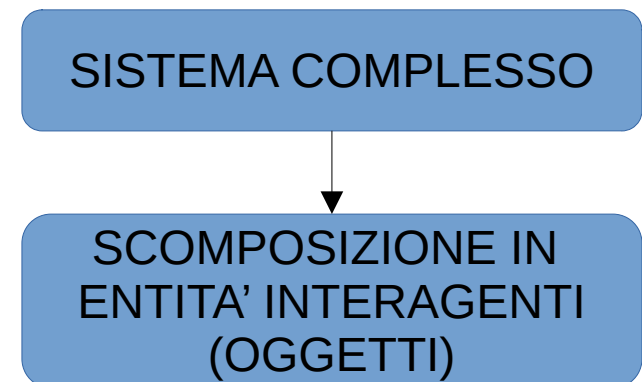
LA PROGRAMMAZIONE AD OGGETTI

- La programmazione ad oggetti rappresenta la naturale evoluzione della programmazione strutturata. Non ci si concentra più solo sulla sequenza di istruzioni, ma sulla progettazione di un sistema composto da oggetti che collaborano tra loro.
- Cambiamento di punto di vista
 - Da: “**Programmare un programma**” (sequenze, funzioni, procedure)
 - A: “**Programmare un sistema**” (insieme di oggetti che interagiscono)
- Questo è un **cambiamento radicale**: non si parte più **dal come** risolvere il problema, ma **dal chi** lo risolve.

PROGRAMMAZIONE STRUTTURATA



PROGRAMMAZIONE AD OGGETTI



FOCUS della OOP

1. Analisi dei dati

- Identificazione delle entità del mondo reale
- Individuazione delle loro caratteristiche (attributi)
- Definizione dei loro comportamenti (metodi)

2. Comportamento del sistema

- Come gli oggetti si comportano
- Come reagiscono agli eventi
- Come modificano il proprio stato interno

3. Interazione tra oggetti

- Gli oggetti collaborano
- Lo fanno tramite scambio di messaggi (invocazione di metodi tra oggetti)

VANTAGGI DELL'OOP

- **MODULARITA':**

Il codice di ogni oggetto è indipendente dal codice degli altri.

- Ogni oggetto può essere scritto, testato e mantenuto separatamente.
- Favorisce lo sviluppo parallelo tra più programmatori.
- Riduce l'impatto degli errori: un problema in un modulo non compromette l'intero sistema.

- **INFORMATION-HIDING:**

Gli oggetti espongono solo ciò che serve: i metodi pubblici.

- I dettagli interni dell'implementazione rimangono nascosti.
- Protegge i dati da modifiche indesiderate.
- Permette di cambiare l'implementazione interna senza modificare il resto del programma.

VANTAGGI DELL'OOP

- **RIUTILIZZO DEL CODICE:**

Se un oggetto esiste già, può essere riutilizzato in nuovi programmi.

- Riduce i tempi di sviluppo.
- Aumenta l'affidabilità (si riutilizzano componenti già testati).
- Favorisce la creazione di librerie e framework.

- **PLUGGABILITY AND DEBUGIND EASE:**

Gli oggetti sono come componenti intercambiabili.

- Se un oggetto crea problemi, può essere sostituito con un altro simile.
- Analogia: come cambiare un pezzo meccanico in una macchina.
- Il debugging è più semplice perché il comportamento è confinato dentro l'oggetto.

VANTAGGI DELL'OOP

- **FACILITÀ DI LETTURA E COMPRENSIONE**

Il codice rispecchia il mondo reale: oggetti, attributi, comportamenti.

- La struttura è più intuitiva.
- Il flusso logico è più chiaro.
- Facilita la collaborazione tra sviluppatori.

- **MANUTENZIONE NEL TEMPO**

La manutenzione è più semplice e meno rischiosa.

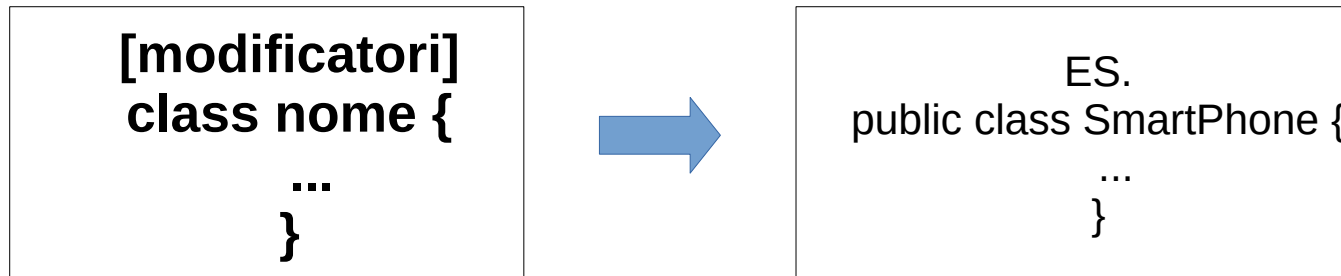
- Le modifiche sono localizzate negli oggetti interessati.
- È possibile estendere il sistema aggiungendo nuovi oggetti senza toccare quelli esistenti.
- Ideale per software che evolvono nel tempo.

LE CLASSI e GLI OGGETTI

- Le **classi** costituiscono l'**elemento base** della programmazione a oggetti.
- La classe è la **descrizione astratta** di un tipo di dato (ADT) e descrive una famiglia di oggetti con *caratteristiche e comportamenti simili*
- Un **oggetto** è un'**istanza** di una determinata classe.
- L'istanza rappresenta un elemento **distinto**, allocato in memoria, con propri attributi appartenenti alla stessa natura di quelli definiti nella classe, ma che possono assumere valori propri e differire da istanza a istanza, e quindi rappresentano **LO STATO** dell'oggetto stesso.
- *Es Pensiamo alle biciclette: hanno due ruote, hanno un colore, hanno una dimensione delle ruote, quindi tutte hanno delle caratteristiche in comune. A livello di programmazione la **bicicletta** genericamente intesa è la **classe**, mentre la **mia bicicletta** è un **oggetto** che appartiene alla classe "bici". Da qui si può dire che la **classe bicicletta** costituisce il **template** di ogni oggetto bicicletta, e che **ogni istanza** è un **elemento concreto in memoria che può assumere valori propri**.*

Creare una classe

La **sintassi** per definire una classe è la seguente:



- I **modificatori di accesso** sono **parole chiave** aggiunte alla classe, o alla dichiarazione dei membri per specificare dove possono essere utilizzati.
- I modificatori **più utilizzati** sono: **Public e Private**.
 - **Public**: i tipi e membri public non hanno limiti di accesso
 - **Private**: accesso consentito SOLO all'interno della classe che definisce l'elemento

Membri di una classe

- Nelle classi dobbiamo distinguere due elementi principali:
 - **Variabili: mantengono lo stato di un oggetto**
 - **Metodi: definiscono il comportamento di un oggetto**

Variabili

Attribuiscono a una classe le **caratteristiche** di cui vogliamo dotarla. Queste possono comprendere tipi primitivi, come int, double, bool, char o anche altri oggetti. La sintassi per dichiarare una variabile è la seguente:

[modificatori] tipo nome

==>

```
class Smartphone
{
    private string marca;
    private string modello;
}
```

NOTA: anche se le variabili possono essere definite public, generalmente sono private; per default private

Metodi

- I metodi rappresentano le **azioni** che è possibile effettuare sugli oggetti, o che un singolo oggetto può fare. La sintassi per scrivere un metodo è la seguente:

```
[modificatori] TipoRitorno NomeDelMetodo (parametri) {  
    ...  
}
```

- I **modificatori** sono identici a quelli delle variabili e mantengono le stesse funzioni. Il **tipo di ritorno** identifica il risultato che l'elaborazione del metodo restituisce. Nel caso questa non produca output, il tipo di ritorno sarà **void**.
- Il metodo può **ricevere** delle variabili che serviranno per poter completare l'elaborazione, queste sono dette **parametri** e sono **opzionali**.
- Ogni metodo che restituisce un tipo (quindi non void) deve includere al suo interno l'istruzione **return**.

```
public string GetDescrizione() {  
    return marca + " " + modello;  
}
```

- NOTA: all'interno del metodo si possono utilizzare i campi della classe come se fossero variabili locali; in effetti l'ambito dei campi si estende all'interno dei metodi della loro classe

CHIAMATA DI METODI

```
public void ImpostaDati(string ma, string mod)
{
    marca=ma;
    modello=mod;
}
```

```
static void Main()
{
    Smartphone phone=new Smartphone();
    phone.ImpostaDati("mia marca", "mio mod");
}
```

COSTRUTTORE

- E' un metodo particolare che permette di definire come deve avvenire la creazione di un oggetto e, quindi, come deve essere istanziato. Qui è bene scrivere tutto il codice che definirà i **valori iniziali** delle variabili d'istanza.

```
public class Smartphone {
    private string marca;
    private string modello;

    public Smartphone() {
        marca = "mia marca";
        modello = "mio modello";
    }
}
```

```
public class Smartphone {
    private string marca;
    private string modello;

    public Smartphone(string m, string mod) {
        marca = m;
        modello = mod;
    }
}
```

LE PROPRIETA'

- Le Property sono state introdotte nel linguaggio C# come scorciatoia sintattica per evitare di scrivere i metodi get/set. Le Property sostituiscono i metodi get/set e nascondono i corrispondenti attributi privati.
- Con le proprietà, siamo in grado di accedere ai campi istanza sia in lettura che in scrittura; questa è la sintassi:

```
public tipo nome_proprietà
{
    get
    {
        return nome_campo;
    }
    set
    {
        nome_campo = value;
    }
}
```

```
class Smartphone
{
    private string mmodello;    //CAMPO

    public string Modello
    {
        get
        {
            return mmodello;
        }
        set{
            mmodello=value;
        }
    }
}
```

La segnatura indica che la proprietà è public, quindi accessibile dal codice esterno alla classe, è dichiarata di un tipo tra quelli consentiti dal linguaggio e che è in stretta relazione con quello del campo istanza a cui farà riferimento ed infine ha un nome.

- All'interno del blocco set è possibile utilizzare la parola chiave **value**, che contiene il valore assegnato alla proprietà.

Per leggere e scrivere una proprietà, si utilizza l'operatore = di assegnazione

```
Smartphone phone=new Smartphone();  
phone.Modello="abc123";
```



*Assegnando un valore abc123 alla proprietà,
all'interno del blocco set, la parola chiave
value conterrà tale valore.*

Al contrario per utilizzare il valore di una proprietà, basta accedere ad essa mediante l'operatore . come un qualsiasi membro

```
Console.WriteLine(phone.Modello);
```

Il risultato potrebbe sembrare molto simile a quello che si otterrebbe creando dei campi con modificatore public. La differenza è che mediante una proprietà è possibile eseguire del codice per controllare come ottenere dei dati privati della classe o per controllare l'impostazione di tali dati, **SENZA** dare un accesso diretto ad essi.

*es. si assegna al campo modello un valore solo se il
valore passato alla proprietà ha una lunghezza minore di
10 caratteri.*

```
public string Modello  
{  
    get {  
        return mmodello;  
    }  
    set {  
        if(value.Length<10)  
            mmodello=value;  
    }  
}
```

Le proprietà possono contenere entrambe le sezioni (get/set) o solo una di esse, rendendole a “**sola lettura**” o “**sola scrittura**”

Una proprietà può restituire **non solo un campo** ma anche un valore basato su un calcolo o su una combinazione di altri campi della classe

```
class CStudente
{
    string mName;    // campo istanza privato della classe
    public string Nome    // definisco la proprietà per il campo "Nome"
    {
        get          // per la lettura
        {
            return mName;
        }
        set          // per l'assegnazione
        {
            mName = value; // assegno un valore stringa al campo
        }
    }

    string mCognome;    // campo istanza privato della classe
    public string Cognome // è dello stesso tipo del campo istanza
    {
        get { return mCognome; }
        set { mCognome = value; }
    }

    public string NomeCompleto // def. la proprietà per il campo "NomeCompleto"
    {
        get
        {
            // per la SOLA lettura
            return mCognome + " " + mName;    // ritorna il nome completo
        }
    }
} // fine classe
```



```
class Program
{
    public static void Main()
    {
        Studente alunno = new Studente();           // istanzio la classe
        alunno.Nome = "Valentino";                   // valorizzo le proprietà dell'oggetto
        alunno.Cognome = "Rossi";
        Console.WriteLine("\nIl nome dell'alunno è {0} {1}",alunno.Nome, alunno.Cognome);

        Console.ReadLine();
    }
} // chiudo il programma
```

Esercizio Guidato

Classe Libro

Creare un nuovo file che contiene la classe **Libro**: questa classe è il modello di ogni libro che si può creare nel programma

- **Libro.cs**: file C# contenente SOLO la classe Libro
- **Attributi** o **Campi**: variabili che rappresentano lo *stato* di un libro
 - titolo, autore, anno, pagine, ...
- **Metodi**: funzioni utilizzabili tramite la variabile *libro* che usano lo stato
- *Costruttore*: metodo con lo stesso nome della classe e usato ogni volta che si crea una variabile (istanza) di tipo libro
- *string Info()*: ritorna informazioni sul libro
- Seguendo le linee della programmazione ad oggetti:
 - Tutti gli attributi hanno il prefisso **private**
 - Tutti i metodi hanno il prefisso **public**

Classe Libro

```
Libro.cs > ...
1  // Classe Libro
2
3  13 references
  public class Libro {
4
5      // Stato di un libro: titolo, autore, anno e numero di pagine
6      2 references
7      private string titolo;
8      2 references
9      private string autore;
10     2 references
11     private int anno;
12     1 reference
13     private int pagine;
14
15     // Costruttore : utilizzato per creare nuovi libri
16     3 references
17     public Libro(string titoloLibro, string autoreLibro, int annoLibro, int numPagine) {
18         this.titolo = titoloLibro;
19         this.autore = autoreLibro;
20         this.anno = annoLibro;
21         this.pagine = numPagine;
22     }
23
24     // Info : Informazioni sul libro
25     0 references
26     public string Info() {
27         return $"{this.titolo}' di {this.autore} (Anno {this.anno})";
28     }
29 }
```

Ogni attributo è preceduto dal suo *tipo* specifico

- Il *costruttore* ha tanti parametri quanti sono gli attributi da inizializzare: ogni volta che si crea un libro si dovranno specificare tutti gli argomenti così da non avere attributi NON inizializzati
- *this* si utilizza per distinguere i parametri dagli attributi e rappresenta l'istanza corrente di una classe (in questo caso di un libro ben preciso)
- Si seguono le regole del CamelCase:
 - Nome del file e della classe: iniziale *maiuscola*
 - Membri *privati*: iniziale *minuscola*
 - Membri *pubblici*: iniziale *maiuscola*

Utilizzo di Libro

Modificare *Program.cs* creando un nuovo libro e mostrando le relative informazioni a video:

```
C# Program.cs > ...  
1  
2  Libro uno = new Libro("1984", "George Orwell", 1949, 333);  
3  
4  Console.WriteLine("Libro n.1: " + uno.Info());
```

Classe Autore

In modo analogo a quanto fatto per la classe *Libro*, introdurre la classe *Autore*: questa classe è il modello di ogni autore che si può creare nel programma

- **Autore.cs**: file C# contenente SOLO la classe Autore
- **Attributi**: variabili che rappresentano lo *stato* di un autore
 - cognome, nome, maschio, data di nascita, elenco libri pubblicati
- **Metodi**: funzioni utilizzabili tramite la variabile *autore* che usano lo stato
 - *Costruttore*: metodo con lo stesso nome della classe e usato ogni volta che si crea una variabile (istanza) di tipo autore
 - *string Info()*: ritorna informazioni sull'autore
- Seguendo le linee della programmazione ad oggetti:
 - Tutti gli attributi hanno il prefisso **private**: usabili SOLO all'interno della classe
 - Tutti i metodi hanno il prefisso **public**: usabili anche all'esterno della classe

Libro e Autore

Una volta introdotta la classe *Autore*, è possibile utilizzarla nella classe *Libro*, modificando il tipo della variabile *autore* da *string* ad *Autore* ma anche cambiando il tipo del parametro nel costruttore di *Libro*:

```
public class Libro {  
  
    // Stato di un libro: titolo, autore ...  
    2 references  
    private string titolo;  
    2 references  
    private Autore autore;  
}
```



Array di Oggetti

Aggiungere alla classe *Autore* un attributo che rappresenta l'elenco dei *Libri* scritti in uno dei seguenti modi:

- **Array di Libri:** `Libro[] libri;` il contenitore avrà però una dimensione fissa assegnata nel costruttore (qui vuoto)

```
private Libro[] libri;  
  
2 references  
public Autore(string cognome, string nome, DateOnly nascita, bool maschio) {  
    { // ...  
  
    this.libri = new Libro[] {};  
}
```

- **Lista di Libri:** `List<Libro> libri;` il contenitore avrà una dimensione dinamica e non fissa (meglio!)

```
private List<Libro> libri;  
  
2 references  
public Autore(string cognome, string nome, DateOnly nascita, bool maschio) {  
    { // ...  
  
    this.libri = new List<Libro>();  
}
```


Collegamento Libro ↔ Autore

Inserire nella classe *Autore* un metodo *Aggiungi* che permette di aggiungere un libro al contenitore dei libri scritti (qui si utilizza un *List<Libro>* con metodo *Add*)

```
public class Autore {  
  
    // Aggiungi : Aggiunge un libro all'elenco degli scritti dall'autore  
    1 reference  
    public int Aggiungi(Libro libroNuovo) {  
        this.libri.Add(libroNuovo);  
        return this.libri.Count;  
    }  
}
```

Questo metodo si può utilizzare nel costruttore del libro per aggiungere la sua nuova istanza (**this**) nell'elenco degli scritti

```
// Costruttore : utilizzato per creare nuovi libri  
3 references  
public Libro(string titoloLibro, Autore autoreLibro, int annoLibro, int numPagine) {  
    // ...  
    autoreLibro.Aggiungi(this);  
}
```

Classe Biblioteca

Creare un nuovo file che contiene la classe *Biblioteca*: di questa classe è creata una sola istanza che contiene tutti gli autori e i relativi libri

- **Biblioteca.cs**: file C# contenente SOLO la classe Biblioteca
- **Attributi**: nome, elenco autori, elenco libri, ...
- **Metodi**:
 - *Costruttore*
 - *ToString()*: ritorna informazioni sulla biblioteca (autori e libri pubblicati)
 - *int AggiungiAutore(Autore autore)*: aggiunge un autore alla biblioteca
 - *AutoriInAnno(int anno)*: ritorna un array degli autori nati nell'anno specificato
 - *LibriInAnno(int anno)*: ritorna un array di libri pubblicati nell'anno specificato
- **Proprietà**: funzioni *get* utilizzabili per fornire valori senza usare attributi interni:
 - *Autori[] ElencoAutori*: array di tutti gli autori presenti
 - *Libri[] ElencoLibri*: array di tutti i libri presenti
 - *int TotalePagine*: la somma di tutte le pagine di tutti i libri

Utilizzo di Biblioteca

Da verificare:

- Come si gestisce l'elenco di tutti gli autori?
- Come si gestisce l'elenco di tutti i libri?
- Come si trovano tutti gli autori nati in un anno?
- Come si trovano tutti i libri pubblicati in un anno?
- Come si sommano tutte le pagine di tutti i libri?

Ci sono varie soluzioni, ma sono tutte ugualmente efficienti?

- Ad esempio per trovare tutti gli autori nati in un anno:

```
// AutoriInAnno : Autori nati nell'anno specificato
0 references
public Autore[] AutoriInAnno(int anno) {
    return this.autori.FindAll(autore => autore.Nascita.Year == anno).ToArray();
}
```

Utilizzo di Biblioteca (2)

Modificare *Program.cs* creando l'unica variabile *biblioteca* a cui associare gli autori e i libri precedentemente creati:

```
Biblioteca biblio = new Biblioteca("ITIS Rossi");
biblio.AggiungiAutore(orwell);
biblio.AggiungiAutore(yourcenar);

Console.WriteLine($"{biblio}\nElenco Autori:");
foreach (Autore autore in biblio.ElencoAutori)
    Console.WriteLine($"- {autore}\n  libri presenti: {autore.LibriPubblicati.Length}");
int idx = 0;
Console.WriteLine("Elenco Libri:");
foreach (Libro liber in biblio.ElencoLibri)
    Console.WriteLine($"{++idx}. {liber}");
```